

# AUTONOMOUS AI WORKFORCE

7-Day Development Workplan • 3-Member Team • Basic + Advanced Project Mode

**Objective:** Deliver a working local-first autonomous workforce MVP in seven days using one shared architecture, fixed interfaces, three ownership areas, Basic Mode for lightweight tasks, and Advanced Project Mode for complex multi-agent projects.

**Architecture basis:** The supplied company architecture defines the core control loop, five Executive Digital Twins, Specialist Agents, Obsidian memory, guardrails, verification/reflection, and a 3-member / 7-day execution target.

## 1. Locked General Architecture

**USER INPUT → PERCEPTION → MODE SELECTOR → MASTER ORCHESTRATOR → MEMORY + PLANNER → CAPABILITY MANAGER → GUARDRAIL → TWIN / SPECIALIST → EXECUTION → OBSERVATION → VERIFICATION.**

**Success:** Verification → Output / Walkthrough → Approved Memory.

**Failure:** Verification → Evidence → Reflection → Re-plan Gate → Planner → Retry → Verification.

**Rule:** Basic and Advanced are two execution paths over the same platform; Advanced adds project initialization, deeper planning and workforce coordination.

## 2. Operating Modes

| Basic Mode                                                                                                                                                                                                                                         | Advanced Project Mode                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Purpose:</b> Fast direct assistance.</p> <p>Input → Perception → Capability → Security → Execution → Verification → Output.</p> <p>Use for transcription, image generation, simple content, document operations and other bounded tasks.</p> | <p><b>Purpose:</b> Complex multi-step projects.</p> <p>Input → Project Initialization → Orchestrator → Memory → Planner → Capability Manager → Security → Workforce → Execution → Verification → Reflection/Re-plan → Walkthrough → Memory.</p> |

## MEMBER 1 — CORE INTELLIGENCE & CONTROL

### 3. Member 1 — Complete Responsibility

**Primary ownership:** the central brain and autonomous control loop. Member 1 makes the system understand the request, create a plan, select capabilities, authorize actions, coordinate execution, verify outcomes and recover from failures.

#### Core modules

- Perception Agent — intent, goals, constraints, required outputs and context.
- Master Orchestrator — task lifecycle, state, routing and coordination.
- Planner — decomposition, dependencies, checkpoints, success criteria and stopping conditions.
- Capability Manager — maps plan steps to capabilities, agents, tools and models.
- Guardrail / Security — authorization, permissions, risk checks and audit events.
- Execution Manager — schedules approved actions and tracks runtime state.
- Verification — objective quality checks and evidence collection.
- Reflection / Re-plan — analyzes verified failure evidence and proposes corrective planning.
- Re-plan Gate — validates evidence support, scope, expected impact and verification criteria.
- Retry Controller — bounded retries and safe stop/escalation.

### 4. Member 1 — Basic Mode

**Path:** Input → Perception → Capability Identification → Security → Specialist/Model → Verification → Output.

**Example:** "Transcribe this audio." Perception identifies the request → capability selected → approved ASR execution → verification → transcript.

**Deliverable:** a stable lightweight control path that every Basic capability can reuse.

### 5. Member 1 — Advanced Project Mode

- Initialize project state: requirements, goals, constraints, success criteria and context.
- Coordinate the complete project lifecycle through the Master Orchestrator.
- Generate structured execution plans and dependencies.
- Route plan steps to capabilities and workforce.
- Apply guardrail/security decisions before execution.
- Track Spawn → Assign → Run → Monitor → Verify → Complete → Terminate.
- Collect objective evidence from execution.
- Trigger Reflection only after a real verification failure.
- Pass the proposed correction through the Re-plan Gate before retry.
- Stop/escalate when recovery is unsupported or retry limit is reached.

### 6. Member 1 — 7-Day Timeline

| Day   | Work                                                                                        |
|-------|---------------------------------------------------------------------------------------------|
| DAY 1 | Backend skeleton, environment, shared schemas, API conventions, logging and task contracts. |
| DAY 2 | Perception + Master Orchestrator + task state and routing.                                  |
| DAY 3 | Planner + Capability Manager + Guardrail/Security.                                          |
| DAY 4 | Workforce routing and Twin/Specialist invocation contracts.                                 |
| DAY 5 | Execution Manager, queues, timeouts and event logging.                                      |
| DAY 6 | Verification, evidence collection, Reflection and Re-plan Gate.                             |
| DAY 7 | Full control-loop integration, stabilization and final demo.                                |

## MEMBER 2 — AI MODELS, TOOLS & SOFTWARE DEVELOPMENT

### 7. Member 2 — Complete Responsibility

**Primary ownership:** the execution layer that performs real work using local AI models and controlled tools.

- Local model runtime / serving on the shared AI server.
- Reasoning and coding model integration through one model-service abstraction.
- Audio/speech model integration for transcription.
- Vision and other model adapters as capabilities are enabled.
- Filesystem, terminal, Git, browser and Docker tool adapters.
- Isolated Software Development workspace.
- Code generation, repository understanding and code modification.
- Build, unit, integration and E2E test execution.
- Security scanning and execution evidence.
- Model routing/loading so large models are not run simultaneously.

### 8. Member 2 — Basic Mode

Implement reliable capability execution for registered local models/tools: **ASR • reasoning • image • document • content • other approved capabilities.**

Every capability should expose the same execution interface so the Orchestrator does not need capability-specific logic.

### 9. Member 2 — Advanced Project Mode

**Flagship capability: SOFTWARE DEVELOPMENT.**

- Requirement arrives as a structured project task.
- Planner/Capability Manager requests Software Development.
- Software Development worker operates in an isolated workspace.
- Frontend, Backend, Database, Testing and Security specialists can be invoked.
- Agents can read/write project files, use approved terminal commands, run Git and Docker workflows.
- Generated changes are tested and produce evidence for Verification.
- Failed tests feed objective evidence back to Member 1's Reflection/Re-plan pipeline.

### 10. Member 2 — 7-Day Timeline

| Day   | Work                                                                                    |
|-------|-----------------------------------------------------------------------------------------|
| DAY 1 | NVIDIA/CUDA/Docker setup on shared server; validate GPU access and local model serving. |
| DAY 2 | Model-service abstraction, structured outputs and coding/reasoning model connectivity.  |
| DAY 3 | Tool interfaces and isolated execution foundation.                                      |
| DAY 4 | Software Development capability and specialist execution contracts.                     |
| DAY 5 | Real code generation/modification, tool control and test execution.                     |
| DAY 6 | Build/unit/E2E/security verification and evidence output.                               |
| DAY 7 | AI performance, execution stability and final integration.                              |

## MEMBER 3 — MEMORY, EXECUTIVE WORKFORCE & PRODUCT

### 11. Member 3 — Complete Responsibility

**Primary ownership:** knowledge, Executive Digital Twins, Specialist workforce registry and product/UI integration.

- Memory Agent + the supplied local Obsidian knowledge base.
- Knowledge retrieval before work and approved memory write after work.
- Controlled external research → evidence validation → approved memory.
- CEO, COO, CTO, CMO and CFO Executive Digital Twins.
- Specialist Agent registry and lifecycle visualization.
- Frontend integration with the existing Command Center / Advanced Workspace.
- Activity feed, project state, walkthrough and final artifact presentation.
- Connect real backend states to the existing UI without redesigning the locked frontend.

### 12. Member 3 — Basic Mode

Maintain the Basic capability registry and connect UI actions to the shared capability/execution contracts.

For ordinary tasks, do **not** force an Executive Twin. A Specialist can be invoked directly when strategic reasoning is not required.

### 13. Member 3 — Advanced Project Mode

**Routing rule:** Capability Manager determines whether strategic reasoning is needed. If yes, activate the appropriate Executive Twin; otherwise route directly to a Specialist.

- CEO — business-level strategic reasoning.
- COO — operational reasoning and workflow.
- CTO — architecture and technical strategy.
- CMO — marketing/content strategy.
- CFO — financial reasoning.
- Twins delegate concrete work to Specialist Agents.
- Specialists execute the actual capability tasks and are terminated when their work is complete.

### 14. Member 3 — Memory Architecture

**Memory Agent → Internal Obsidian + Controlled External Research → Evidence Validation → Approved Knowledge → Obsidian.**

External research must not become trusted company memory merely because a model retrieved it. The Memory workflow should preserve source/evidence context and only write approved knowledge.

### 15. Member 3 — 7-Day Timeline

| Day   | Work                                                                         |
|-------|------------------------------------------------------------------------------|
| DAY 1 | Validate existing frontend, routes, Advanced Mode and integration contracts. |
| DAY 2 | Connect Task Composer/project UI to backend task and status APIs.            |
| DAY 3 | Obsidian Memory Agent: read/search/write abstractions.                       |
| DAY 4 | CEO/COO/CTO/CMO/CFO Twin registry + Specialist Agent registry/lifecycle.     |
| DAY 5 | Workforce + Memory integration; real activity states in UI.                  |
| DAY 6 | Verification/retry/reflection UI + final walkthrough/artifact views.         |
| DAY 7 | Frontend, Twins, Memory and end-to-end demo integration.                     |

## 16. Shared Collaboration Contract

**Golden Rule: IMPLEMENTATION CAN DIFFER. INTERFACES CANNOT.**

All three developers work from one Git repository using feature branches. The architecture contracts are locked first. Internal implementation choices may differ, but the inputs/outputs and lifecycle states must remain identical.

| Shared Contract | Purpose                                                            |
|-----------------|--------------------------------------------------------------------|
| Task            | User request, mode, requirements, constraints, status and result.  |
| Plan            | Structured steps, dependencies, capabilities and success criteria. |
| Agent           | Identity, role, capability, model, tools, state and progress.      |
| Capability      | Standard description and execution interface.                      |
| Tool            | Name, permissions, inputs, outputs and execution result.           |
| Memory          | Query, retrieved context, evidence and approved write.             |
| Execution       | Action, workspace, tool/model call, logs and result.               |
| Verification    | Checks, evidence, pass/fail and reason.                            |
| Reflection      | Failure evidence, root-cause hypothesis and proposed correction.   |
| Event           | Timestamped system/agent activity for observability.               |

## 17. Integrated 7-Day Milestones

| Day   | Team-level output                                                                    |
|-------|--------------------------------------------------------------------------------------|
| DAY 1 | Environment + contracts + frontend/backend skeleton.                                 |
| DAY 2 | Perception + orchestration + local model service + frontend API connection.          |
| DAY 3 | Planning + capability/security + Obsidian Memory + execution tools.                  |
| DAY 4 | Workforce routing + Executive Twins + Specialists + Software Development capability. |
| DAY 5 | Real autonomous execution + model/tool integration + memory.                         |
| DAY 6 | Verification + evidence + Reflection + Re-plan + Retry.                              |
| DAY 7 | Full integration + stabilization + final walkthrough + demo.                         |

## 18. Reliability & Hallucination Controls

- **Evidence before Re-plan:** Reflection receives test output, stack traces, logs, diffs, runtime output and verification results.
- **Re-plan Gate:** validate evidence support, scope, expected impact and verification criteria.
- **Independent verification:** the model generating a fix must not be the sole authority declaring success.
- **Bounded recovery:** recommended MVP maximum of 3 recovery attempts, then stop/escalate.
- **Memory safety:** external research is validated before becoming approved Obsidian knowledge.
- **Security:** approved autonomous actions can proceed without unnecessary interruption; sensitive/high-risk actions remain guarded.

## 19. Development Hardware Allocation

| System                                                     | Owner    | Use                                                                              |
|------------------------------------------------------------|----------|----------------------------------------------------------------------------------|
| MacBook M1 • 8GB                                           | Member 1 | Backend, orchestration, APIs, tests and control logic.                           |
| HP Victus • RTX 3050 • 16GB                                | Member 2 | AI/tool development, local testing and Docker; not the primary heavy-model host. |
| Lenovo IdeaPad Slim 5 • 24GB                               | Member 3 | Frontend, Memory/Obsidian, Twins and integration.                                |
| Shared AI Server • RTX 3090 24GB • 64–128GB RAM • 2TB NVMe | Team     | Local model serving and heavier shared inference.                                |

## 20. Final End-to-End Demonstration

**Example request:** “Build a complete company landing website using the company's existing information.”

1. Voice/Text input reaches Perception.
2. System detects software-development intent.
3. Advanced Project Mode initializes the project.
4. Memory retrieves relevant Obsidian/company context.
5. Planner creates a structured execution plan.
6. Capability Manager selects Software Development, Testing and Security capabilities.
7. Guardrail validates the planned actions.
8. CTO Twin activates for architecture when strategic reasoning is needed.
9. Frontend, Backend, Database, Testing and Security specialists are spawned.
10. Execution occurs inside the controlled workspace.
11. Verification runs objective checks.
12. A controlled failure produces evidence.
13. Reflection analyzes the evidence.
14. Re-plan Gate validates the proposed correction.
15. Planner issues the revised plan.
16. Agents retry.
17. Independent verification passes.
18. Workers complete and terminate.
19. Final walkthrough presents files, artifacts, changes and verification.
20. Approved project state is written to Obsidian.

## 21. Day-7 Definition of Done

- ✓ Basic Mode works for direct capability tasks.
- ✓ Advanced Project Mode initializes and tracks a complex project.
- ✓ Voice/text input reaches Perception.
- ✓ Local model service is reachable and used for selected capabilities.
- ✓ Planner produces a structured plan.
- ✓ Capability Manager selects the appropriate workforce.
- ✓ Executive Twin activates when strategic reasoning is required.
- ✓ Multiple specialists can be spawned, run, verified and terminated.
- ✓ Software Development performs a real controlled task.
- ✓ Verification produces objective evidence.
- ✓ Failure can trigger evidence-based Reflection → Re-plan → Retry.
- ✓ Final walkthrough and artifacts are visible.
- ✓ Approved project state can be stored in the supplied Obsidian knowledge base.
- ✓ Typecheck, production build and core integration tests pass.

## 22. Scope Control for the 7-Day MVP

**Critical path:** Basic Mode + Advanced Project Mode + Software Development + Audio Transcription + Obsidian Memory + autonomous execution/recovery loop.

**Capability strategy:** register the broader tool/model inventory behind the same Capability interface. Production-grade implementation of every media/productivity capability is not required to prove the core workforce loop in one week.

**Final engineering principle:** keep the core agents stable; add new capabilities through the Capability Manager, Specialist registry, model adapters and tool interfaces rather than modifying the central control loop.